Data Structures and Algorithms

Theoretical Concepts & Implementations

This document provides a theoretical foundation, step-by-step implementation guide, complexity analysis, and visual representations for core Data Structures and Algorithms.

---

1. Arrays

Concept Theory: An array is a linear data structure that stores elements of the same data type in contiguous memory locations. Because the memory is contiguous, any element can be accessed in constant time `O(1)` using its index.

Visual Representation:

```
Memory Address:  100   104   108   112   116
Index:          [ 0 ] [ 1 ] [ 2 ] [ 3 ] [ 4 ]
Value:          | 10  | 20  | 30  | 40  | 50  |
```

Time Complexity:

- **Access:** O(1)
- **Search:** O(n)
- **Insertion/Deletion:** O(n) (requires shifting elements)

Step-by-Step Implementation & Internal Working: In Java, standard arrays are fixed in size. Dynamic arrays (`ArrayList`) resize automatically by creating a new larger array (usually 1.5x to 2x the size) and copying over elements when the capacity is reached.

```java
public class DynamicArray {
    private int[] data;
    private int size;
    private int capacity;

    public DynamicArray(int capacity) {
        this.capacity = capacity;
        this.data = new int[capacity];
        this.size = 0;
    }

    public void add(int element) {
        if (size == capacity) resize(); // Step 1: Check capacity
        data[size++] = element;         // Step 2: Insert & increment size
    }

    private void resize() {
        capacity *= 2;                  // Double the capacity
        int[] newData = new int[capacity];
        for (int i = 0; i < size; i++) newData[i] = data[i]; // Copy elements
        data = newData;                 // Replace old array
```

```
    }
}
```

---

2. Linked Lists

Concept Theory: A Linked List is a linear data structure where elements (nodes) are not stored contiguously. Each node points to the next node forming a chain. This allows for constant time `O(1)` insertions and deletions at known positions, but `O(n)` sequential access.

Visual Representation:

```
Head
  |
[ Data: 10 | Next ] ---> [ Data: 20 | Next ] ---> [ Data: 30 | Next ] ---> null
```

Time Complexity:

- **Access / Search:** O(n)
- **Insertion/Deletion at Head:** O(1)
- **Insertion/Deletion at End/Middle:** O(n) (requires traversal)

Step-by-Step Implementation & Internal Working:

```java
public class LinkedList {
    // Step 1: Define the Node class
    class Node {
        int val;
        Node next;
        Node(int val) { this.val = val; }
    }

    private Node head;

    // Step 2: Implement insertion at head
    public void addFirst(int val) {
        Node newNode = new Node(val);
        newNode.next = head; // Point new node to current head
        head = newNode;      // Make new node the head
    }

    // Step 3: Implement traversal
    public void printList() {
        Node curr = head;
        while (curr != null) {
            System.out.print(curr.val + " -> ");
            curr = curr.next;
        }
        System.out.println("null");
    }
}
```

---

3. Stacks

Concept Theory: A Stack follows the Last-In-First-Out (LIFO) principle. Think of it like a stack of plates—you can only add or remove the top plate.

Visual Representation:

```
        Push (30)        Pop () -> 30
            |              ^
            v              |
        | 30 |         |    |
        | 20 |         | 20 |
        | 10 |         | 10 |
        +----+         +----+
```

Time Complexity:

- **Push / Pop / Peek:** O(1)

Step-by-Step Implementation & Internal Working: Stacks can be implemented using Arrays or Linked Lists.

```java
public class StackArray {
    private int[] arr;
    private int top;

    public StackArray(int capacity) {
        arr = new int[capacity];
        top = -1; // Indicates empty stack
    }

    public void push(int val) {
        if (top == arr.length - 1) throw new StackOverflowError();
        arr[++top] = val; // Increment top and insert
    }

    public int pop() {
        if (top == -1) throw new EmptyStackException();
        return arr[top--]; // Return value and decrement top
    }
}
```

---

4. Queues

Concept Theory: A Queue follows the First-In-First-Out (FIFO) principle. Like a line at a ticket counter, the first person to join is the first to be served.

Visual Representation:

```
Enqueue(40) -----> [ 40 | 30 | 20 | 10 ] -----> Dequeue() -> 10
                   Rear             Front
```

Time Complexity:

- **Enqueue / Dequeue:** O(1)

Step-by-Step Implementation: Queues are best implemented with a Linked List to maintain O(1) dequeue operations, avoiding the O(n) shifting required in arrays.

```
public class QueueLinkedList {
    class Node {
        int processId; Node next;
        Node(int id) { processId = id; }
    }

    private Node front, rear;

    public void enqueue(int id) {
        Node node = new Node(id);
        if (rear == null) {
            front = rear = node;
            return;
        }
        rear.next = node; // Link the old rear to the new node
        rear = node;      // Update rear pointer
    }

    public int dequeue() {
        if (front == null) return -1;
        int id = front.processId;
        front = front.next; // Move front pointer forward
        if (front == null) rear = null; // Queue is now empty
        return id;
    }
}
```

---

5. Trees (Binary Search Tree)

Concept Theory: A Binary Search Tree (BST) is a hierarchical structure where each node has at most two children. It maintains the BST Property: left child's value < parent's value, and right child's value > parent's value. This enables fast searching similar to binary search in arrays.

Visual Representation:

```
        10
      /    \
    5       15
   / \     /  \
  3   7   12   18
```

Time Complexity:

- **Search / Insert / Delete:** O(log n) average, O(n) worst case (unbalanced).

Step-by-Step Implementation:

```
public class BST {
    class Node {
        int val;
        Node left, right;
        Node(int v) { val = v; }
    }

    Node root;

    public void insert(int val) {
        root = insertRec(root, val);
    }

    private Node insertRec(Node root, int val) {
        if (root == null) return new Node(val); // Found correct empty spot

        if (val < root.val)
            root.left = insertRec(root.left, val); // Go left
        else if (val > root.val)
            root.right = insertRec(root.right, val); // Go right

        return root;
    }
}
```

---

6. Graphs

Concept Theory: A Graph models pairwise relationships between objects. It consists of Vertices (nodes) and Edges (connections). It can be Directed/Undirected and Weighted/Unweighted.

Visual Representation (Adjacency List):

```
Graph:          Adjacency List:
 (0)---(1)      0: [1, 2]
  |   /         1: [0, 2]
  |  /          2: [0, 1, 3]
 (2)---(3)      3: [2]
```

Time Complexity:

- **BFS / DFS Traversal:** O(V + E) where V=vertices, E=edges.

Step-by-Step Implementation:

```
import java.util.*;

public class Graph {
    // Array of lists where index is vertex, list contains neighbors
    private List<List<Integer>> adj;
```

```java
    public Graph(int vertices) {
        adj = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adj.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adj.get(source).add(destination);
        adj.get(destination).add(source); // Undirected graph
    }

    public void bfs(int startNode) {
        boolean[] visited = new boolean[adj.size()];
        Queue<Integer> queue = new LinkedList<>();

        visited[startNode] = true;
        queue.add(startNode);

        while (!queue.isEmpty()) {
            int curr = queue.poll();
            System.out.print(curr + " ");

            for (int neighbor : adj.get(curr)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.add(neighbor);
                }
            }
        }
    }
}
```

---

7. Hashing (Hash Tables)

Concept Theory: A Hash Table maps keys to values using a Hash Function. The hash function converts a key into a bucket index where the value is stored. Collisions (two keys mapping to the same bucket) are handled conventionally via Chaining (linked lists at each bucket) or Open Addressing.

Visual Representation (Separate Chaining):

```
Keys: "apple", "banana", "peach"
Hash Function -> Index

[ 0 ] -> null
[ 1 ] -> ["apple", 5] -> ["peach", 2] -> null  (Collision handled by chaining)
[ 2 ] -> null
[ 3 ] -> ["banana", 10] -> null
```

Time Complexity:

- **Search / Insert / Delete:** O(1) average, O(n) worst case (severe collisions).

Internal Working & Implementation:

```
public class SimpleHashTable {
    class HashNode {
        String key; int value; HashNode next;
        HashNode(String k, int v) { key = k; value = v; }
    }

    private HashNode[] buckets;
    private int capacity;

    public SimpleHashTable(int capacity) {
        this.capacity = capacity;
        buckets = new HashNode[capacity];
    }

    private int getBucketIndex(String key) {
        return Math.abs(key.hashCode()) % capacity; // Convert hash to index
    }

    public void put(String key, int value) {
        int index = getBucketIndex(key);
        HashNode head = buckets[index];

        // 1. Check if key exists, update value
        while (head != null) {
            if (head.key.equals(key)) { head.value = value; return; }
            head = head.next;
        }

        // 2. Insert at head of linked list in the bucket
        HashNode newNode = new HashNode(key, value);
        newNode.next = buckets[index];
        buckets[index] = newNode;
    }
}
```

---

8. Heaps (Priority Queues)

Concept Theory: A Heap is a complete binary tree that satisfies the Heap Property. In a Max-Heap, the parent is strictly greater than its children; in a Min-Heap, the root is the smallest element. They are widely used to maintain a dynamic top-K list or implement Priority Queues.

Because it's a complete tree, it's typically implemented using an array, where for any index `i`, left child is `2i+1` and right child is `2i+2`.

Visual Representation (Min-Heap):

```
Tree View:                Array Representation:
       5                  Index:   0    1    2    3    4    5
     /   \                Value: [ 5 ][ 10 ][ 15 ][ 20 ][ 30 ][ 40 ]
   10     15
  /  \    /
 20  30  40
```

Time Complexity:

- **Insert / Extract Min:** O(log n)
- **Get Min (Peek):** O(1)

Internal Working:

```java
import java.util.Arrays;

public class MinHeap {
    private int[] heap;
    private int size;

    public MinHeap(int capacity) { heap = new int[capacity]; }

    public void insert(int val) {
        if (size == heap.length) heap = Arrays.copyOf(heap, size * 2);
        heap[size] = val; // Add to end
        heapifyUp(size++); // Bubble up to fix heap property
    }

    private void heapifyUp(int index) {
        int parentIndex = (index - 1) / 2;
        // Swap if parent is greater than current child
        while (index > 0 && heap[parentIndex] > heap[index]) {
            int temp = heap[index];
            heap[index] = heap[parentIndex];
            heap[parentIndex] = temp;

            index = parentIndex;
            parentIndex = (index - 1) / 2;
        }
    }
}
```

---

9. Tries (Prefix Trees)

Concept Theory: A Trie is an N-ary tree used for storing and rapidly searching strings (like dictionary words, autofill prefixes). Instead of storing the string at a node, the path traced from root to node represents the string.

Visual Representation:

```
Root
 |-- 'c'
 |    |-- 'a'
 |         |-- 't' (isEnd=true) -> "cat"
 |         |-- 'r' (isEnd=true) -> "car"
 |-- 'd'
      |-- 'o'
           |-- 'g' (isEnd=true) -> "dog"
```

Time Complexity:

- **Search / Insert Prefix:** O(L) where L is string length. Fast and unaffected by the number of words!

Internal Working:

```java
public class Trie {
    class TrieNode {
        TrieNode[] children = new TrieNode[26]; // 26 letters a-z
        boolean isEndOfWord = false;
    }

    private TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int index = c - 'a';
            // Create path if it doesn't exist
            if (curr.children[index] == null) {
                curr.children[index] = new TrieNode();
            }
            curr = curr.children[index];
        }
        curr.isEndOfWord = true; // Mark string termination
    }
}
```

---

10. Strings

Concept Theory: A String is a sequence of characters, typically implemented as an array of characters under the hood. In Java, Strings are immutable, meaning once a String object is created, its value cannot be changed. Any modification results in a new String object. For mutable sequences of characters, `StringBuilder` or `StringBuffer` are used.

Visual Representation:

```
String: "hello"
Index:    [ 0 ] [ 1 ] [ 2 ] [ 3 ] [ 4 ]
Char:     | 'h' | 'e' | 'l' | 'l' | 'o' |
```

Time Complexity:

- **Access:** O(1)
- **Search (Substring):** O(N * M) naive, or O(N) with algorithms like KMP
- **Concatenation:** O(N + M) (creating a new string of combined length)

Step-by-Step Implementation & Internal Working: In Java, String manipulation using `+` in loops is inefficient due to immutability. `StringBuilder` provides an efficient, mutable alternative.

```
public class StringConcepts {
    public static void main(String[] args) {
        // String Immutability
        String str1 = "hello";
        String str2 = str1;
        str1 = str1 + " world"; // Creates a new String object

        // Efficient way: StringBuilder
        StringBuilder sb = new StringBuilder("hello");
        sb.append(" world"); // Modifies the existing array

        // Step 1: Using char array to build strings manually
        char[] chars = {'h', 'e', 'l', 'l', 'o'};
        String fromChars = new String(chars);
    }

    // Step 2: Custom String Builder representation
    static class SimpleStringBuilder {
        private char[] value;
        private int count;

        public SimpleStringBuilder(int capacity) {
            value = new char[capacity];
        }

        public void append(String str) {
            if (str == null) return;
            int len = str.length();
            ensureCapacity(count + len);
            str.getChars(0, len, value, count); // Copy characters
            count += len;
        }

        private void ensureCapacity(int minimumCapacity) {
            if (minimumCapacity > value.length) {
                int newCapacity = value.length * 2 + 2;
                if (newCapacity < minimumCapacity) newCapacity = minimumCapacity;
                char[] newValue = new char[newCapacity];
                System.arraycopy(value, 0, newValue, 0, count);
                value = newValue; // Replace with expanded array
            }
        }
```

```
        public String toString() {
            return new String(value, 0, count);
        }
    }
}
```

Generated mathematically and algorithmically to serve as a comprehensive preparation guide for software engineering interviews.